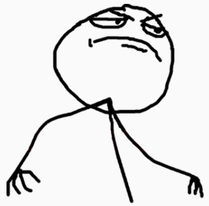
`jq`: deal with `JSON` like a boss

Xavier "Pamplemousse" Maso

2026-03-12



BORDEAUX

JSON

🍊 Xavier Maso (il/lui)

🛠️ RSSI en prestation au Laboratoire de l'Innovation de l'ANSSI

💻 git.sr.ht/~pamplemousse[1]
🫂 mamot.fr/@Pamplemouss_
🏠 www.xaviermaso.com

[1] Vous y trouverez d'ailleurs les slides, et l'environnement utilisé pour les démos : git.sr.ht/~pamplemousse/jq_like_a_boss.

Présentation

—

Table des matières

1 Intro 3
2 Pré-requis... `JSON` 6
3 101 10
3.1 Lancer `jq`, filtres 11
3.2 Extraire des informations 12
3.3 Chainer des opérations 14
3.4 Opérations 15
3.5 Produire des valeurs 16
4 Boucler 17
4.1 Avec des objets 19
4.2 Avec des chaines de charactères 20
4.3 Trier et aggréger 21
4.4 Prédicats 22
5 CLI 23
6 Programmation (fonctionnelle) 25
6.1 Transformer 26
6.2 Fonctions 27
6.3 Variables 28
7 Autres trucs utiles 29
7.1 Astuces 30
7.2 Aller trop loin 31
8 Crédits 33

Table des matières

—

1 Intro

> jq is like `sed` for JSON data - you can use it to slice and filter and map and transform structured data with the same ease that `sed`, `awk`, `grep` and friends let you play with text.
>
> [...]
>
> jq can mangle the data format that you have into the one that you want with very little effort, and the program to do so is often shorter and simpler than you'd expect.
>
> — https://jqlang.org/

1 Intro

—

manuels

- https://jqlang.org/manual
- https://manpages.org/jq

playground

- https://play.jqlang.org

s'entraîner

- https://exercism.org/tracks/jq

1 Intro

—

manuels

- https://jqlang.org/manual
- https://manpages.org/jq

playground

- https://play.jqlang.org

s'entraîner

- https://exercism.org/tracks/jq

> fin de la préz, merci, de rien, au revoir messieurs-dames
>
> — ~ Perceval

—

2 Pré-requis… `JSON`

2 Pré-requis… JSON

> JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language
>
> — https://www.json.org/

2 Pré-requis… JSON

—

2 Pré-requis… JSON

```
{
  "name": {
    "last": "Maso",
    "first": "Xavier",
    "handle": "Pamplemousse"
  },
  "yearOfBirth": 1990,
  "opinions": {
     "fanOf": ["jq", "elm", "nix"],
     "hatesAI": true
  },
  "profiles": [{
    "network": "Mastodon",
    "url": "https://mamot.fr/@pamplemouss_/"
  }]
}
```


2 Pré-requis… JSON

—

2 structures :

- collection de paires clef-valeurs (objet, record, stuct, dictionnaire, hashed table, etc.)
- liste ordonnée (tableau, vecteur, liste, séquence)

objet, tableau, chaine, nombre, “true”, “false”, “null”

récursif

Des sous-ensembles intéressants et/ou amusants :

- geoJSON : encodage de données géospatiales,
- SARIF : format standardisé de sortie d'outils d'analyse statique de code,
- jsonresume : représentation des données d'un CV,
- autres…

… et si on a pas de JSON 🤯 ?

- jc : outil en CLI qui permet de convertir les sorties de différents outils en CLI,
- yq : wrapper de jq pour gérer du YAML/XML/TOML.

2 Pré-requis… JSON

—

3 101

3.1 Lancer jq, filtres

> by default, jq reads a stream of JSON entities (including numbers and other literals) from stdin
>
> [...]
>
> A jq program is a “filter”: it takes an input, and produces an output.
>
> [...]
>
> Filters can be combined in various ways
>
> — man jq


```
< examples/pamplemousse.json jq '.'
```

3.2 Extraire des informations

identité	`.`
index de tableau	`.[N]`
index d'objet	`.clef` `.["clef"]`
slice	`.[N:M]`
valeur alternative	`chemin // défaut`
filtre les éléments d'un objet ou d'une liste	`pick(un_chemin, ...)`

"path expressions" complexes `.clef.autreClef.[0]`

les indices d'un objet	`keys` `keys_unsorted`
taille d'une liste	`length`

3.3 Chainer des opérations

pipe |

L'opérateur qui fera de vous un plombier heureux, avec un magnifique sourire.

3.4 Opérations

arithmétique	`+, -, *, /, %`
booléenne	`and, or, not`
comparaisons	`==, !=, <, >, <=, >=`
interpolation de chaines	`"...\(filtre)..."`


```
jq '<value> | not
```

3.5 Produire des valeurs

Créer des…

	N
scalaires	`"..."`
listes	`[...]`
objets	`{...}`

4 Boucler

appliquer un filtre aux éléments d'une liste	`map(f)`
appliquer un filtre aux valeurs d'un objet	`map_values(f)`
désimbriquer des listes de listes	`flatten`
filtrer une valeur	`select(b)`
aggréger	`reduce .[] as $e (valeur; f)`

4 Boucler

—

Et donc, pour filtrer une liste :

```
map(select(b))
```

transformer un objet en liste	`to_entries`
transformer une liste en objet	`from_entries`
`to_entries | map(f) | from_entries`	`with_entries(f)`


```
echo '{"a": "b", "c": "d"}' | jq 'with_entries(.key |= "_"
+ . | .value |= "_" + .)'
```

découper une chaine de charactères en liste	`split(c)`
transformer une liste en chaine de charactères	`join(c)`

4.3 Trier et aggréger

trier une liste	`sort`
trier un objet	`sort_by(f)`
dédupliquer les éléments d'une liste	`unique` `unique_by`

inclusion	`contains(valeur)`
présence d'un champ dans l'objet	`has("clef")`
au moins une valeur doit être vraie	`any` `any(condition)`
tout doit être vrai	`all` `all(condition)`

inclusion : pour les chaines de charactères, les listes, et les objets

5 CLI

sortie brute	`--raw-output / -r`
sans pretty-print	`--compact-output / -c`
charger plusieurs valeurs `JSON`	`--slurp / -s`
tri les objets par clefs	`--sort-keys / -S`
charger le code `jq` depuis un fichier	`--from-file / -f`


```
$ (echo '{"name": "Pamplemousse"}'; sleep 2; echo '{"name":
"Xavier"}') | jq '.name'
"Pamplemousse"
"Xavier"
$ (echo '{"name": "Pamplemousse"}'; sleep 2; echo '{"name":
"Xavier"}') | jq --slurp 'map(.name)'
[
  "Pamplemousse",
  "Xavier"
]
```

6 Programmation (fonctionnelle)

suppression	`del(.clef)` `del(.[N])`
assignation	`chemin = valeur`
mise à jour	`chemin |= expression`

> Values in jq are always immutable. Internally, assignment works by [computing] replacement values for . that have had all the desired assignments applied to ., then outputting the modified value.
>
> — `man jq`

`|=` est l'update assignment, et permet de mettre à jour en fonction de la valeur précédente

- nommer des filtres, et les réutiliser
- ~lazy : les arguments sont passés comme des filtres, pas comme des valeurs !

définition	`def nom_de_fonction: f;`
appel	`nom_de_fonction`
définition avec paramètre	`def nom_de_fonction(p): f;`
appel avec paramètre	`nom_de_fonction(v)` `f | nom_de_fonction`


```
jq -nfr ./examples/functions.jq`
```

Généralement : pas besoin.

> In jq, all filters have an input and an output, so manual plumbing is not necessary to pass a value from one part of a program to the next. Many expressions, for instance `a + b`, pass their input to two distinct subexpressions (here a and b are both passed the same input), so variables aren't usually necessary in order to use a value twice.
>
> — `man jq`

> The expression `exp as $x | ...` means: for each value of expression exp, run the rest of the pipeline with the entire original input, and with `$x` set to that value.
>
> — `man jq`

7 Autres trucs utiles

rien (même pas `null`)	`empty`
`map(.clef | select(. != null))`	`map(.clef // empty)`
concatène plusieurs filtres	`f1, f2`
formatter la sortie en CSV	`@csv`
gérer les dates	`fromdate`, `todateiso8601`, `fromdateiso8601`, `strftime`

exploration en profondeur	`..`
lire le type d'une valeur	`valeur | type`
expressions régulières	`test`, `match`, `scan`
gestion des erreurs	`try-catch`
streaming	`--stream`
50 nuances de récursion	`while`, `until`, `repeat`, `recurse`, `walk`
Manipuler des « Path Expressions » 🤯🙀	`path`, `getpath`, `setpath`, `delpaths`

!= de l'évaluation dynamique

Merci, questions ?

Merci, questions ?

—

8 Crédits

8 Crédits

- typst - Un système de composition de documents moderne, puissant, et facile à apprendre.
- typix - Compilation déterministe de Typst avec Nix.
- touying - Création de présentations en Typst.
- Nix/NixOS - Un gestionnaire de paquet, et de configuration système, déclaratif.
- opendata.bordeaux-metropole.fr - Des sources de données ouvertes avec lesquelles jouer.

8 Crédits

—